Bachelor Thesis in Computer Science
April 2024

Automated Detection of Fake News in Natural Language Processing: A Comparative Study of TF-IDF and Lexical-Based Stance Detection with Logistic Regression

Venkata Vishnu Vardhan Reddy Gurram

Jyothi Madhurya Nalam

Faculty of Faculty, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Faculty at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Bachelor Thesis in Computer Science. The thesis is equivalent to Weeks weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:
Author(s):
Venkata Vishnu Vardhan Reddy Gurram
E-mail: vegu23@student.bth.se

Jyothi Madhurya Nalam
E-mail: jyna23@student.bth.se

University advisor:
Dr.Lawrence Henesey, Ph.D.
Department of computer science

Faculty of Faculty
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background The rapid growth of social media in today's digital age has led to an alarming rise in fake news, impacting individuals, organizations, and society. The ease of sharing information on social media platforms has facilitated the dissemination of misinformation, posing risks to public opinion, societal norms, and even public safety. Detecting fake news accurately is essential to combating this issue and upholding the credibility of information sources.

Objectives This study aims to compare the effectiveness of Term Frequency-Inverse Document Frequency (TF-IDF) and Lexical-Based Stance Detection with Logistic Regression in detecting fake news. By evaluating the performance of these methods through metrics such as accuracy, precision, recall, and F1-score, the study seeks to provide insights into improving fake news detection systems and promoting information reliability.

Methods The research follows a general experimentation approach, manipulating variables (model types) and observing outcomes to determine the optimal method for fake news detection. Data preprocessing techniques are applied to extract lexical features from the text data. A dataset will be acquired from Kaggle containing labeled news articles with stances categorized as agree, disagree, discuss, or unrelated. Lexical-Based Stance Detection with Logistic Regression and Logistic Regression with TF-IDF models are then trained and evaluated for their effectiveness in classifying the stance of news articles and sentences.

Results The evaluation of the Lexical-Based Stance Detection and Logistic Regression with TF-IDF models reveals their performance in detecting fake news based on lexical features. Metrics such as accuracy, precision, recall, and F1-score are used for comparison. The Logistic Regression model with TF-IDF achieved an accuracy of 81.75%, while the Lexical-Based Stance Detection Logistic Regression model achieved an accuracy of 77.81%.

Conclusions In conclusion, this study contributes to the combat of the pervasive issue of fake news by exploring the effectiveness of TF-IDF and Lexical-Based Stance Detection with Logistic Regression. By comparing the performance of these two models, we gain valuable insights into the power of TF-IDF in capturing the nuances of textual data. The TF-IDF model, with its ability to weigh the importance of words, emerged as the superior approach in this research, achieving higher accuracy and precision in detecting fake news.

Keywords: Fake news detection; Machine learning; Feature extraction; Logistic Regression ; Stance Detection

Acknowledgments

We want to express our sincere gratitude and appreciation to our supervisor, Dr. Lawrence Hennessy, for his invaluable guidance, support, and contributions throughout our thesis project. His expertise, unwavering support, and insightful feedback were instrumental in shaping our research and academic journey. We would also like to extend our heartfelt thanks to our families and friends for their understanding, patience, and encouragement during this endeavor. Their presence in our lives and their contributions to academic and personal growth are deeply appreciated.

Authors:

Venkata Vishnu Vardhan Reddy Gurram

Jyothi Madhurya Nalam

List of Acronyms

NLP	Natural Language Processing
TF-IDF	Term Frequency-Inverse Document Frequency
BoW	Bag-Of-Words
SVM	Support Vector Machine
LR	Logistic Regression
ANN	Artificial Neural Network
NFND	Novel Fake News Detection

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The surge of online social media platforms in recent years has revolutionized how people connect and communicate. These platforms allow users to share information, expand their social networks, and stay updated on current trends and events. However, the rise of social media has also led to the spread of dubious and misleading content, often called "fake news" [42]. The explosion of information in today's digital world, while empowering us with knowledge, has also created an environment where fake news and misinformation can thrive.

The rapid rise of social media platforms has created a double-edged sword: increased information access coupled with the rapid spread of fake news. This misinformation can have real-world consequences, making automatic phony news detection a critical challenge in Natural Language Processing (NLP) [27]. With the overwhelming amount of online content, NLP-based detection offers a practical solution for online platforms, reducing the need for extensive human intervention and mitigating the spread of fake news.

Stance detection is a fundamental task in Natural Language Processing (NLP) that involves determining a position toward a specific target, such as a topic, claim, or event. This task has gained prominence due to its crucial role in various applications, including fake news detection, claim validation, and argument search [39]. In stance detection, the goal is to understand the relationship between two texts on a similar topic. The task is to determine the stance or position of the headline relative to the article's content. Importantly, the headline and the article may or may not address the same topic. Thus, each headline-article pair's relative stance must be classified as unrelated, discuss, agree, or disagree [26].

Feature-based models in stance detection leverage lexical features and machine learning to predict the stance of a news article toward its corresponding headline [18]. These features will then train the logistic regression model for stance classification on the labeled, preprocessed data.

The Term Frequency-Inverse Document Frequency (TF-IDF) model is a prominent and well-established technique in Natural Language Processing (NLP) for extracting features from text data, especially in the context of fake news detection. TF-IDF measures the importance of a term within a document relative to its frequency across a corpus of documents [11]. Specifically, the TF-IDF model assigns higher weights

or significance to terms that appear frequently within a particular document. This weighting scheme helps highlight the distinctive or characteristic words or phrases that may indicate the document's content or nature.

Misinformation and fake news can have significant real-world consequences. It can influence public opinion, shape political agendas, impact economic markets, and even lead to harmful actions or decisions. For example, during the COVID-19 pandemic, misinformation about the virus and treatments confused and hindered public health efforts. Fake news can also fuel social divisions, exploit vulnerabilities, and erode trust in legitimate news sources.

The motivation for this research stems from the need to address the challenges posed by misinformation and its potential impact. The objective is to develop effective methods for detecting and mitigating the spread of fake news, specifically focusing on lexical-based stance detection and logistic regression with TF-IDF. By comparing these approaches, we aim to enhance the accuracy and reliability of fake news detection systems.

This stance detection method was chosen [2]because it empowers human fact-checkers by automatically classifying articles as agreeing, disagreeing, or discussing a claim, quickly retrieving relevant information, and analyzing arguments from various perspectives for validity assessment. The motivation for selecting logistic regression with TF-IDF [36] lies in its effectiveness in handling large volumes of data and its ease of updating the model with new data.

For this thesis project, we will utilize the Fake News Challenge dataset available on Kaggle [2]. The dataset is meticulously organized into two distinct CSV files: train-bodies.csv and train-stances.csv. Each instance in the dataset comprises a headline, the body of the news article, and the stance it takes. The stances are categorized into four types: unrelated, discuss, agree, and disagree.

To normalize the text data, we conducted data preprocessing. This process involved removing non-word characters, converting all text to lowercase, eliminating isolated lowercase letters and extra spaces, removing punctuation, special characters, and numbers, handling contractions, and removing stop words. This preprocessing step prepares the data for further analysis tasks, like feature extraction.

1.1 Ethical, Societal and Sustainability aspects

Ethical concern

For this research, we're using publicly available information from Kaggle [2]. While this ensures accessibility, it's important to acknowledge that it might contain personal details. We take data privacy seriously and will be meticulous in handling this

information. To ensure fairness and minimize bias in our analysis, we'll carefully curate the dataset, anonymize sensitive information, and conduct thorough bias checks.

Societal aspects

Research on detecting fake news in natural language processing (NLP) is crucial due to the increasing spread and impact of misinformation on social media platforms. By automating the identification of fake news, this research promotes information integrity, protects public safety, enhances media literacy, optimizes resource allocation, and preserves trust in online platforms. This project aims to tell the difference between real news and fake news. We'll be using two different methods, called logistic regression and TF-IDF, to see which one works best. We're comparing these two technologies to ensure our detection system is accurate and fair, which is crucial for keeping information reliable.

Sustainability aspects

The development of fake news detection systems has a substantial environmental impact. Training and running machine learning classifiers requires immense energy and storage space. This energy is often derived from fossil fuels, which release harmful greenhouse gases into the atmosphere. In addition, the storage and processing of large datasets can lead to increased electricity consumption at data centers, further exacerbating the environmental footprint of these systems. Ultimately, by empowering individuals to distinguish between fact and fiction through these tools, we can strengthen trust in the media and promote a more transparent information environment.

1.2 Aim and objectives

1.2.1 Aim

This research investigates the effectiveness of two methods for detecting fake news: (1) utilizing a TF-IDF model, and (2) employing a stance detection technique with a lexical-based approach in logistic regression classification.

1.2.2 Objectives

- Acquire a dataset from Kaggle containing labeled news articles with stances categorized as agree, disagree, discuss, or unrelated. Preprocess the data to extract pertinent features utilizing techniques such as bag-of-words (BoW) and n-gram modeling while addressing any noise or inconsistencies present.

- Employ a lexical-based approach to determine the stance of the text regarding a particular topic or claim. Utilize sentiment lexicons and dictionaries to identify sentiments within the text.

- Develop and train a machine learning classifier to predict the text stance based on the extracted lexical features. Assess the classifier's performance using accuracy, precision, recall, and F1-score metrics.

- Evaluate the effectiveness of logistic regression and the TF-IDF model in detecting fake news.

1.3 Problem Statement

The proliferation of fake news on social media platforms poses significant risks to public opinion, societal norms, and public safety. Despite various efforts to combat misinformation, there remains a need for more effective and reliable methods to detect fake news. This research addresses this gap by comparing the effectiveness of TF-IDF and Lexical-Based Stance Detection with Logistic Regression in detecting fake news.

1.4 Research questions

RQ1: In the context of tackling misinformation and upholding information integrity, how do logistic regression and the TF-IDF model compare in their performance and effectiveness in classifying the stance of news articles and sentences using lexical features for fake news detection systems?

Motivation For the evaluation of classifier performance, various metrics, including accuracy, precision, recall, and F1-score, will be taken into account. Accuracy assesses the overall correctness of predictions, while precision and recall gauge the accuracy in identifying fake and real news articles, respectively. The F1-score, being a weighted average of precision and recall, provides a comprehensive measure of classifier effectiveness.

1.5 Scope

This thesis investigates the effectiveness of two methods for combating the spread of fake news: lexical-based stance detection and logistic regression with TF-IDF. By comparing these approaches using a dataset from Kaggle, the project aims to identify the most suitable method for fake news detection. The expected outcome is to contribute to developing more effective fake news detection systems, ultimately promoting information accuracy and reliability. The research will encompass data preprocessing, feature extraction, model training, and evaluation.

1.6 Outline

- **Chapter 1: Introduction:** This chapter will provide an overview of fake news detection, including the aims, objectives, and research question.

- **Chapter 2: Background:** This chapter will provide a comprehensive overview of the background information relevant to fake news detection and machine learning.
- **Chapter 3: Related Work:** This chapter will delve deeper into the existing research on fake news detection, exploring various methods employed and their success rates.
- **Chapter 4: Methodology:** This chapter will outline the general implementation details of the TF-IDF and stance detection approach used in this research.
- **Chapter 5: Results and Analysis:** This chapter will present the findings of our investigation, comparing the performance of both methods in detecting fake news. We will analyze the results.
- **Chapter 6: Discussion:** This chapter will provide a comprehensive discussion of the results, relating them to the research questions and objectives.
- **Chapter 7: Conclusion:** This chapter will summarize the key findings of the research, and propose potential directions for further exploration.

1.7 Input and Output Parameters

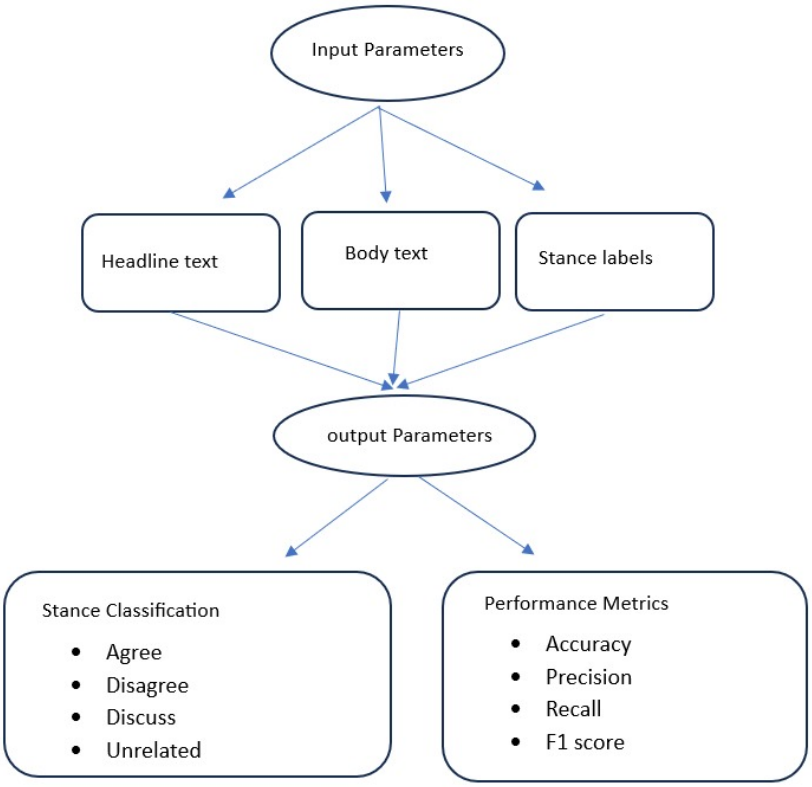

Figure 1.1: Input and Output Parameters

Input:

The input to the system is news articles, which include the headlines and the body text of the articles.

Output:

The system's output is the predicted stance label for each input news article, indicating whether the article agrees, disagrees, discusses, or is unrelated to the given headline, as well as the accuracy, precision, recall, and F1 score.

Chapter 2

Background

2.1 NLP

Natural Language Processing (NLP) plays a vital role in developing automated systems for detecting fake news. NLP techniques allow computers to understand and process human language, enabling them to analyze text data and identify linguistic cues that may indicate deception in the context of fake news detection [20]. To enhance the precision and effectiveness of these NLP tasks, including indexing, topic modeling, text classification, and information retrieval, a crucial pre-processing step involves eliminating uninformative words, commonly known as "stopwords" [7].

Natural Language Processing (NLP) is critical in detecting fake news and misinformation, especially on online social media platforms where false information can spread rapidly with significant real-world consequences. The impact of fake news can be far-reaching, influencing political and social discourse, spreading conspiracy theories, and even causing harm to individuals and communities. By integrating NLP with machine learning techniques, especially supervised learning, we can create powerful systems that automatically detect fake news by analyzing text and identifying deceptive patterns.

2.1.1 NLP Techniques for Fake News Detection

Text Classification: Text classification algorithms are used to categorize text into predefined classes or categories automatically. In the context of fake news detection, text classification can be employed to classify news articles or social media posts as either fake or genuine. Machine learning algorithms, such as Support Vector Machines, and Naive Bayes, and deep learning models like Convolutional Neural Networks are commonly used for this task.

Sentiment Analysis: Sentiment analysis involves analyzing text to determine the sentiment or emotional tone expressed by the author. In fake news detection, sentiment analysis can help identify negative or misleading sentiments that may indicate the presence of fake news. Techniques such as lexicon-based approaches, rule-based systems, machine learning algorithms, and deep learning models are used to analyze the sentiment of text.

Stance Detection: Stance detection aims to determine the stance or opinion of an author towards a particular topic or entity. In fake news detection, stance detection

can be useful in identifying whether a news source or social media user supports, opposes, or is neutral toward a specific claim or news story. Machine learning algorithms, including SVM, logistic regression, and deep learning models like long-short-term memory networks, are commonly employed for stance detection tasks.

Social Network Analysis: Social network analysis examines the relationships and interactions between entities in a social network. In the context of fake news detection, social network analysis can help identify suspicious patterns of information propagation, influential spreaders, and potential fake news sources. Techniques such as centrality measures, community detection, and network embedding are used to analyze the structure and dynamics of social networks.

2.1.2 TF-IDF

Term Frequency-Inverse Document Frequency (TF-IDF) is a statistical measure used to evaluate the importance of a word in a document relative to a collection of documents [40]. It is widely used in text mining and information retrieval to transform text into a numerical representation. TF-IDF measures the importance of a term within a document relative to its frequency across a corpus of documents. Specifically, the TF-IDF model assigns higher weights or significance to terms that appear frequently within a particular document. This weighting scheme helps highlight the distinctive or characteristic words or phrases that may indicate the document's content or nature.

2.1.3 Lexical-Based Stance Detection

Lexical-based Stance detection involves analyzing the lexical features of a text to determine its stance towards a specific target [37]. This method uses sentiment lexicons and dictionaries to identify sentiments within the text. In stance detection, the goal is to understand the relationship between two texts on a similar topic. The task is to determine the stance or position of the headline relative to the article's content. Importantly, the headline and the article may or may not address the same topic. Thus, each headline-article pair's relative stance must be classified as unrelated, discuss, agree, or disagree.

2.1.4 chi-squared test

The chi-squared test is a widely used statistical test that assesses the independence of two variables in a contingency table. It is commonly applied to categorical data to determine if there is a significant relationship between the variables [29]. The test calculates a chi-squared statistic, which can be compared to a critical value to make inferences about the population.

The chi-squared test is based on the chi-squared distribution, which is a theoretical probability distribution. The test compares the expected frequencies (based on the assumption of independence) with the observed frequencies in the data. The chi-squared statistic is calculated as the sum of the squared differences between the expected and observed frequencies, each divided by the expected frequency.

The formula for the chi-squared statistic is as follows [9]:

$$(\chi^2) = \sum \frac{(O - E)^2}{E} \tag{2.1}$$

$$O = \text{Observed frequency} \tag{2.2}$$

$$E = \text{Expected frequency} \tag{2.3}$$

The chi-squared test is often used in feature selection to identify the most informative features. By applying the test to each feature individually, we can assess the significance of the relationship between the feature and the target variable [6]. Features with higher chi-squared statistics indicate a stronger association with the target and are considered more informative. The chi-squared test is based on the assumption of independence, which means that the two variables being tested should be independent of each other. In other words, the occurrence of one variable should not influence or be influenced by the occurrence of the other variable. If this assumption is violated and the variables are not truly independent, it can lead to inaccurate results from the chi-squared test. Specifically calculated chi-squared statistic may be inflated, leading to an increased probability of rejecting the null hypothesis (that the variables are independent) when it should not be rejected.

2.1.5 N-gram

N-gram models are statistical language modeling tools designed to predict the probability of natural word sequences. These models aim to assign higher probabilities to word sequences that occur frequently and lower probabilities to sequences that do not occur. The fundamental idea behind N-gram models is to capture the underlying patterns and structures in natural language by considering the context of consecutive words [41]. N-gram models can be particularly useful for fake news detection by analyzing the linguistic patterns and word sequences present in news articles or social media posts. Fake news often exhibits distinct linguistic characteristics, such as the use of sensational or emotionally charged language, biased or exaggerated claims, and specific word choices or phrases aimed at misleading readers. By training N-gram models on a corpus of known fake and genuine news articles, these models can learn to recognize and assign lower probabilities to word sequences and patterns that are more commonly associated with fake news.

2.2 ML

Machine learning (ML) is a fascinating field of artificial intelligence (AI) that empowers computers to learn and evolve without being explicitly programmed [24]. Instead of following predetermined instructions, ML algorithms use data and experience to improve their performance over time. This dynamic and adaptive nature of ML has revolutionized numerous industries, from healthcare and prediction to semantic Analysis and language processing [35].

Many machine learning tools are used to tackle various problems, and some of the most common ones include linear classifiers, logistic regression, naive Bayes, support vector machines (SVMs), decision trees, random forests, and k-nearest neighbors (k-NN). These tools can also be combined with techniques like AdaBoost and bagging to improve performance [35].

- **Supervised learning:** This ML type involves training a model on labeled examples, where the input data is associated with correct outputs. The goal is to learn the underlying patterns and make accurate predictions on new, unseen data. Examples include Linear Regression, Decision Trees, and Neural Networks.

- **Unsupervised Learning:** Unsupervised learning deals with unlabeled data, aiming to discover hidden patterns, relationships, and insights within the data. Clustering and dimensionality reduction are common techniques used in this type of ML.

- **Semi-supervised Learning:** Semi-supervised learning combines a small amount of labeled data with a larger set of unlabeled data. This approach leverages the benefits of both supervised and unsupervised learning, improving the model's performance and reducing the need for extensive labeled data.

- **Reinforcement Learning:** Reinforcement learning involves an agent interacting with its environment, learning through trial and error to achieve a goal. The agent receives feedback through rewards or penalties, which it uses to improve its decision-making process. This type of ML has been successful in game-playing and robotics.

2.3 Logistic Regression

Logistic Regression (LR) is a well-established statistical technique known for its ability to generate probability models. Logistic regression has found extensive application in information retrieval and has been a subject of investigation in the realm of machine learning [15]. It is well-suited for binary classification.

LR's fundamental strength lies in its capacity to model the relationship between a set of input variables and a binary outcome variable. LR employs a logistic function to transform the linear combination of input variables into a probability value between 0 and 1. This probability represents the likelihood of the binary outcome occurring, making it a valuable tool for classification and prediction tasks [16].

Logistic regression assumes a linear relationship between the input features and the log odds of the output variable. If the relationship between the text features and the stance/fake news classification is highly non-linear, logistic regression may struggle to capture the complex patterns effectively. More advanced non-linear models like neural networks or kernel methods may be better suited in such cases.

2.4 Evaluation Metrics

Accuracy: Accuracy refers to the overall correctness of the model's predictions. It is calculated as the ratio of correctly predicted instances to the total number of instances in the dataset.

Precision: Precision, on the other hand, focuses on the positive predictions made by the model. It is calculated as the ratio of true positive predictions to the total number of positive predictions made by the model.

Recall: Recall measures the proportion of correctly predicted positive instances (agree or disagree) out of all actual positive instances.

F1-Score: The F1-Score is the harmonic mean of precision and recall, providing a single metric that balances both concerns.

Macro-Average: Calculate metrics independently for each class and take the unweighted mean. This treats all classes equally.

Weighted Average: Calculate metrics independently for each class and take a weighted mean based on class frequencies. This gives more weight to larger classes.

Chapter 3

Related Work

Vasu Agarwal et al. [10] research delves into the critical issue of fake news detection using advanced natural language processing (NLP) and machine learning (ML) techniques. They emphasize the pressing need to combat the widespread dissemination of misinformation on digital platforms. The study provides a comprehensive overview of previous efforts in fake news detection, exploring a range of approaches from data mining to neural networks. By employing the LIAR dataset, the authors meticulously train and evaluate five different classifiers, including Naïve Bayes, Logistic Regression, and Support Vector Machine (SVM). Notably, SVM and logistic regression emerge as the top-performing classifiers, with SVM exhibiting slightly higher accuracy and overall performance.

The research conducted by Kanika Jindal et al. [19] addresses the pressing issue of fake news proliferation, particularly in the digital age where social media platforms play a significant role in disseminating information. The study focuses on leveraging machine learning algorithms to detect misleading information accurately. Utilizing the count Vectorizer technique for feature extraction, the researchers evaluate various classifiers including Naïve Bayes, Decision Tree, Support Vector Machine (SVM), and Random Decision Forest. Results indicate that the Random Decision Forest classifier achieves the highest accuracy of 98.49%, outperforming other classifiers. This underscores the effectiveness of ensemble learning techniques for mitigating misinformation.

Karishnu Poddar et al. [30] investigates different machine learning algorithms' efficacy in identifying fake news. They address the growing concern of fake news dissemination, particularly on social media platforms, emphasizing the need for computational models to tackle this issue effectively. The study utilizes datasets from Kaggle and other sources, focusing on the content of news articles for classification. Various machine learning algorithms, including Naive Bayes, Support Vector Machine (SVM), Logistic Regression, Decision Trees, and Artificial Neural Networks (ANN), are evaluated for their performance in fake news detection. Results indicate that SVM with TF-IDF Vectorizer achieves the highest accuracy in fake news detection, with an accuracy score of 92.8%. Logistic regression also performs well, yielding an accuracy score of 91.6%. Conversely, decision trees exhibit lower accuracy throughout the study, with a maximum accuracy of 82.5%. It concluded that the study finds that SVM with TF-IDF vectorizer emerges as the most effective model for fake news detection in the analyzed dataset.

In 2021, Vinit Kumar et al. [21] highlighted the critical necessity of combating misinformation on social media platforms. Using binary classification, the research introduces methodologies to differentiate between counterfeit and genuine news articles while validating the credibility of news sources. Essential aspects of the methodology include aggregating data from diverse online platforms, preprocessing involving data cleansing and TF-IDF vectorization, and the utilization of classifiers such as Naïve Bayes, Logistic Regression, and Random Forest. The study outlines challenges associated with handling unstructured social media data and underscores the significant role of data cleansing in enhancing model accuracy. Logistic regression emerges as the most effective classifier, achieving a 72% accuracy rate in identifying fake news.

Jasmine Shaikh et al. [33] addresses the critical issue of fake news detection, employing Support Vector Machine (SVM), Naïve Bayes, and Aggressive Classifier techniques. Their study highlights the significance of accurately discerning fake news amidst its widespread impact on various sectors, including politics and education, particularly in the context of social media. Through a comprehensive literature review, they explore existing computational models and linguistic analyses used to automatically identify fake news, such as N-gram analysis and SVM. In the methodology, Shaikh and Patil detail the preprocessing steps applied to text data, including stop word removal and stemming, alongside feature extraction using the term frequency-inverse document frequency (TF-IDF). By implementing three classification algorithms and splitting the dataset for training and testing, they achieve notable results, with SVM demonstrating the highest accuracy of 95.05%.

Dr. M. Rajeswari et al. from PSGR Krishnammal College for Women in Coimbatore, India [32], focuses on fake news detection using a logistic regression algorithm with machine learning techniques. The study addresses the challenge of identifying fake news circulated through social media platforms like WhatsApp, Facebook, and Twitter. The proposed model utilizes machine learning and natural language processing to aggregate news and determine its authenticity. Through logistic regression, the model achieves an impressive accuracy level of 97.21%. This work adds to the existing literature on fake news detection and provides valuable insights into the effectiveness of logistic regression in addressing this critical issue.

Leela Siva Rama Krishna et al. [38] from Saveetha School of Engineering investigated the effectiveness of Logistic Regression (LR) for fake news detection, comparing it to the Support Vector Machine (SVM) algorithm. Their study employed datasets of real and fake news (N=311 for both LR and SVM) and achieved an accuracy of 95.12% with LR, outperforming SVM's accuracy of 91.68%. LR also demonstrated superior precision (93.62%) compared to SVM (89.20%). Motivated by limitations in existing detection systems, the authors conducted a comprehensive analysis, highlighting LR's potential as a supervised learning technique for effective fake news identification. Their work proposes a novel fake news detection (NFND) mechanism based on LR's performance. It offers a valuable contribution to the field and a reference point for future research in fake news detection.

Shete et al. [34] explore machine learning methods for fake news detection, focusing on logistic regression. Their work comprehensively reviews preprocessing techniques like lemmatization, tokenization, and stop word removal, while also comparing logistic regression with algorithms like K-Nearest Neighbor, Naive Bayes, and Support Vector Machine. They describe their dataset in detail, including source, size, and attributes. Following data cleaning, they analyze the performance of their logistic regression model, achieving an accuracy of 80% and a precision of 86%. Their analysis includes insightful discussions on the confusion matrix, ROC curve, and the impact of additional features such as the website interface, language translation capabilities, Twitter bot integration, and dynamic labeling of CSV files.

The literature review reveals a range of studies focused on detecting fake news using machine learning techniques. Various classifiers, feature extraction methods, and datasets have been explored, providing valuable insights into effective approaches for combating misinformation. Among the classifiers investigated, Support Vector Machine (SVM) and Logistic Regression have often achieved higher accuracy, demonstrating their potential to distinguish between genuine and fake news. Feature extraction techniques, such as TF-IDF and Count Vectorizer, play a crucial role in capturing the importance and frequencies of words within the text, contributing to more informed decision-making by the classifiers.

The reviewed studies have employed datasets from diverse sources, including the LIAR dataset, Kaggle datasets, and data collected from social media platforms. This variety of sources reflects the real-world nature of fake news dissemination and the need for robust detection systems that can adapt to different contexts. Evaluation metrics such as accuracy, precision, recall, and F1-score have been commonly used to assess the performance of the proposed methods, providing a comprehensive understanding of their effectiveness.

The literature review reveals that while various studies have explored the use of TF-IDF, Lexical-Based Stance Detection, and Logistic Regression individually for fake news detection, there is a lack of research integrating these techniques within a unified framework. Combining the strengths of TF-IDF in capturing word importance and Lexical-Based Stance Detection in identifying linguistic cues could potentially enhance the performance of Logistic Regression classifiers for fake news detection. This thesis aims to address this research gap by investigating the interplay between these techniques and their impact on the accuracy, precision, recall, and F1-score of fake news detection systems.

The motivation for this research arises from the pressing need to combat the spread of fake news and its detrimental impact on society. Misinformation has the potential to influence public opinion, shape political landscapes, and even endanger lives. By combining TF-IDF and Lexical-Based Stance Detection techniques, we expect to develop a more robust and effective approach to identify and mitigate the dissemination of misleading information. Understanding how these techniques influence the

accuracy, precision, recall, and F1 score of classifiers will provide valuable insights for improving the performance of fake news detection systems.

Author	**Approach**	**Classifiers Evaluated**	**Key Findings**
Vasu Agarwal et al.	Fake news detection using NLP and ML	Naive Bayes, Logistic Regression, SVM	SVM and Logistic Regression perform best, with SVM slightly more accurate.
Kanika Jindal et al.	Leveraging machine learning algorithms	Naive Bayes, Decision Tree, SVM, Random Decision Forest	Random Decision Forest achieve the highest accuracy (98.49%), highlighting ensemble learning effectiveness.
Karishnu Poddar et al.	Evaluating different ML algorithms	Naive Bayes, SVM, Logistic Regression, Decision Trees, ANN	SVM with TF-IDF Vectorizer outperforms others.
Vinit Kumar et al.	Binary classification and data aggregation	Naive Bayes, Logistic Regression, Random Forest	Logistic regression emerges most effective (72% accuracy).
Jasmine Shaikh et al.	Support Vector Machine, Naive Bayes, Aggressive Classifier	SVM, Naive Bayes, Aggressive Classifier	SVM shows the highest accuracy (95.05%).
Dr. M. Rajeswari et al.	Fake News Detection using logistic regression and ML techniques	Logistic Regression	Logistic regression achieves impressive accuracy (97.21%).
Leela Siva Rama Krishna et al.	Comparing Logistic Regression and SVM	Logistic Regression, SVM	Logistic Regression outperforms SVM with higher accuracy and precision.
Shete et al.	Exploring Logistic Regression and other algorithms	Logistic Regression, K-Nearest Neighbor, Naive Bayes, SVM	Logistic Regression model achieves 80% accuracy and 86% precision.

Table 3.1: Summary of Related Work

Chapter 4

Method

This thesis project adopts a systematic approach, known as general experimentation, to explore and evaluate the effectiveness of different models in detecting fake news. The Fake News Challenge dataset, sourced from Kaggle, serves as our foundation. Our methodology begins with acquiring this dataset, providing a solid starting point for our research. The raw data undergoes thorough preprocessing, including cleaning, transformation, stop word removal, and tokenization. This step ensures data consistency and readiness for analysis. Following data preprocessing, the dataset is divided into two subsets: training data and testing data.

The training data, comprising 60% of the dataset, is utilized to train machine learning models, while the remaining 20% is allocated for validation and the final 20% serves as the testing data for evaluating model performance. Two distinct models are trained: logistic regression with TF-IDF model and lexical-based stance detection. These models are then evaluated using the testing data, with performance metrics such as accuracy, precision, recall, and F1-score calculated to assess their effectiveness.

Finally, the efficiency and effectiveness of the entire process are assessed to ensure that the chosen model not only accurately detects fake news but also operates efficiently. The visual representation of the methodology is depicted in the diagram below, providing a clear overview of the steps involved in the research process.

4.1 Working Environment

The code was developed and executed in a Python programming environment, leveraging the capabilities of various libraries and frameworks. Python, known for its simplicity and versatility, offers a wide range of tools and packages that facilitate data analysis, machine learning, and natural language processing tasks.

Pandas: Pandas is a powerful data manipulation and analysis library, particularly suited for tabular data. It offers efficient data structures like DataFrames and Series, along with a rich set of functions for data handling, transformation, exploration, and aggregation. Pandas simplifies the process of loading, cleaning, and analyzing structured data [13].

NumPy: NumPy is a fundamental library for numerical computing and array manipulation. It provides efficient data structures like arrays, along with a collection of mathematical functions for performing element-wise operations, matrix calculations, and other numerical computations. NumPy is essential for efficient mathematical operations and array-based computations [1].

Matplotlib: Matplotlib is a plotting library used for creating visualizations, such as plots, charts, and graphs. It enables the creation of informative and visually appealing graphs, allowing for effective communication of insights and trends in the data. Matplotlib provides a wide range of plot types, customization options, and tools for data visualization [12].

Seaborn: Seaborn builds on top of Matplotlib, providing additional functionality for creating aesthetically pleasing and informative statistical visualizations. It simplifies the process of creating complex visualizations, such as heatmaps, box plots, and violin plots, while offering pre-defined themes and color palettes. Seaborn enhances the visual representation of data, making it easier to interpret and communicate insights [4].

NLTK: The Natural Language Toolkit (NLTK) is a comprehensive suite of libraries and programs for symbolic and statistical natural language processing. It offers a wide range of functionalities, including tokenization (splitting text into words or sentences), stemming (reducing words to their base or root form), stopword removal (eliminating common words that may not carry significant meaning), and more. NLTK provides a solid foundation for text preprocessing and analysis [23].

sklearn: Scikit-learn is a robust machine-learning library that provides a wide range of algorithms and tools for model training, evaluation, and preprocessing. It offers functions for data splitting, feature extraction, model selection, and evaluation metrics. Scikit-learn is widely used for building and evaluating machine learning models [17].

TfidfVectorizer: This module within scikit-learn transforms text data into TF-IDF vectors. TF-IDF captures the importance of words in a document relative to the en-

tire corpus, weighing them based on their frequency and rarity. It is particularly useful for text classification and information retrieval tasks [22].

DictVectorizer: The DictVectorizer module transforms dictionary-like data into a machine-learning-friendly format. In the code, it is used to convert the dictionary of lexical features, extracted from the article bodies, into a numerical representation suitable for machine learning algorithms [5].

LogisticRegression: Logistic Regression is a popular machine learning algorithm used for binary or multi-class classification tasks. It models the relationship between input features and a binary or categorical target variable. In the code, LogisticRegression is used to train a model that predicts the stance ('agree', 'disagree', 'discuss', 'unrelated') based on the extracted features [16].

By using these libraries, the code can efficiently preprocess text data, extract relevant features, train machine learning models, and evaluate their performance. The choice of Python and its extensive ecosystem of libraries streamline the development process, making it easier to tackle complex tasks in natural language processing and machine learning.

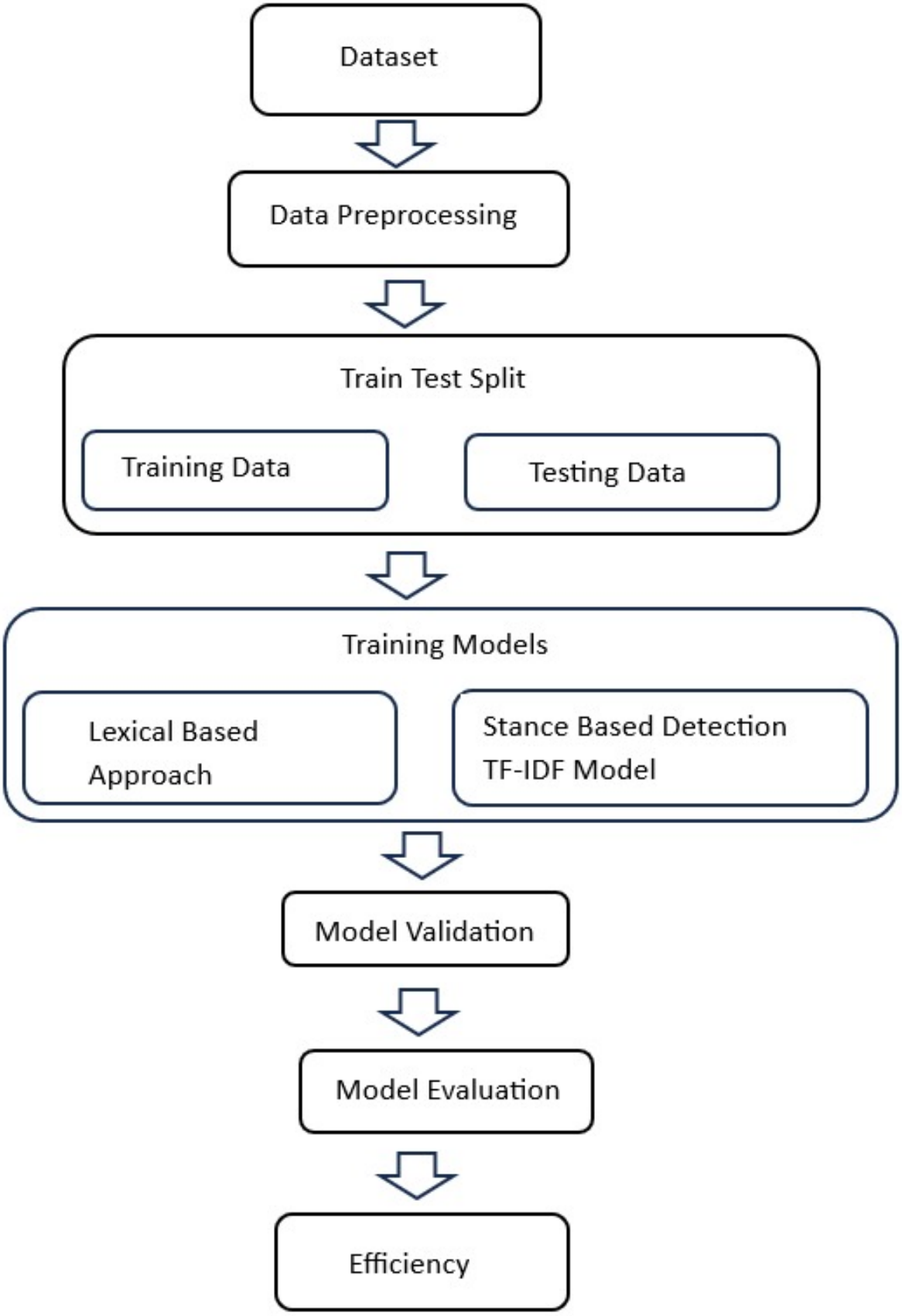

Figure 4.1: Experimentation procedure

4.2 Dataset

This thesis will utilize the Fake News Challenge dataset, available on Kaggle [3]. This dataset contains headlines, body text, and stance labels. Stance labels categorize instances as unrelated, discussing, agreeing, or disagreeing. It consists of two CSV files: "train-bodies.csv" holds article text with unique identifiers, while "train-stances.csv" matches headlines with body text and stance labels. Upon analyzing the stance label distribution within the dataset, it becomes apparent that the largest portion of instances comprising 73.13% of the data are classified as "unrelated." The distribution further reveals that "discuss" accounts for 17.82%, "agree" for 7.36%, and "disagree" for the remaining 1.68%. This distribution is visually represented in Table 4.1 provided below:

Agree: Headline and article say the same thing.

Disagree: Headline and article argue opposite sides.

Discuss: The headline and article touch on the same topic, but the article stays neutral.

Unrelated: Article talks about something completely different from the headline.

Rows	Unrelated	Discuss	Agree	Disagree
49972	73.13%	17.82%	7.36%	1.68%

Table 4.1: Distribution of Stance Labels in the Dataset

	A	B	C
1	Headline	Body ID	Stance
2	Police find mass graves with at least '15 bodies' near Mexi	712	unrelated
3	Hundreds of Palestinians flee floods in Gaza as Israel oper	158	agree
4	Christian Bale passes on role of Steve Jobs, actor reported	137	unrelated
5	HBO and Apple in Talks for $15/Month Apple TV Streamin	1034	unrelated
6	Spider burrowed through tourist's stomach and up into hi	1923	disagree
7	'Nasa Confirms Earth Will Experience 6 Days of Total Dark	154	agree
8	Accused Boston Marathon Bomber Severely Injured In Pri	962	unrelated
9	Identity of ISIS terrorist known as 'Jihadi John' reportedly	2033	unrelated
10	Banksy 'Arrested & Real Identity Revealed' Is The Same Hc	1739	agree
11	British Aid Worker Confirmed Murdered By ISIS	882	unrelated
12	Gateway Pundit	2327	discuss
13	Woman detained in Lebanon is not al-Baghdadi's wife, Ira	1468	agree
14	Kidnapped Nigerian schoolgirls: Government claims cease	1003	unrelated
15	No, that high school kid didn't make $72 million trading st	2132	unrelated
16	Soon Marijuana May Lead to Ticket, Not Arrest, in New Yo	47	discuss
17	Vandals add rude paint job to $2.5m Bugatti (but luckily fc	615	unrelated

Figure 4.2: Data Visualization

4.3 Pre-Processing

In the data preprocessing stage, we transformed the Kaggle dataset from its incomplete and inconsistent state into a format suitable for machine learning models.

Conversion to Lowercase: All text characters were converted to lowercase letters, ensuring consistency and eliminating case sensitivity.

Stopword Removal: Stopwords, such as 'of', 'the', 'and', and 'an', were removed as they have minor importance in terms of features and are irrelevant to the specific task.

Stemming: Stemming was applied to reduce words to their base or root form, consolidating similar words and aiding in feature extraction [28].

Tokenization: Tokenization involves breaking down text into individual words or tokens, enabling further analysis and processing [25].

Substitution of Non-Alphanumeric Characters: Any non-alphanumeric character was substituted with a space, effectively removing punctuation and special characters.

Single-Character Word Removal: Single-character words were removed, specifically targeting those surrounded by spaces and at the beginning of the text.

Whitespace Standardization: Consecutive whitespace characters were replaced with a single space, ensuring a clean and uniform representation of whitespace.

After executing the preprocessing steps, the preprocess function was applied to the 'Headline' and 'articleBody' columns of the merged_df DataFrame using the apply() method. This ensured that each value in these columns underwent the same preprocessing transformations, resulting in a consistent and standardized dataset ready for further analysis or modeling.

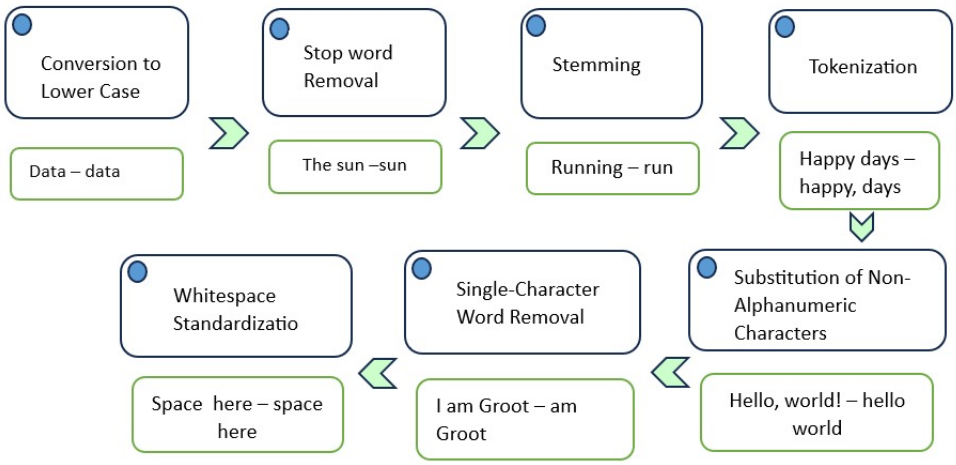

Figure 4.3: Pre-processing of text data

4.4 Classifiers

4.4.1 Lexical-based approach

In our approach, we utilize a lexical feature extraction function to capture the essence of the article bodies. The function is a crucial part of our methodology. It plays a critical role in analyzing the text data by examining the occurrence of specific words associated with different stance categories. where each stance category, such as "agree," "disagree," "discuss," and "unrelated," is matched with a list of relevant words. This dictionary serves as a reference guide, allowing us to identify and count the occurrences of these stance-related words within the text. By analyzing the frequency of stance-related words, both individually (word counts) and in sequences (bigrams), our function aims to identify the underlying lexical patterns that differentiate between various stances in the article text. This feature extraction step transforms the textual data into a format conducive to machine learning algorithms, enabling us to build an effective stance detection model.

Bag-of-words

By focusing on the frequency of words within a document and disregarding their order or grammatical structure, the Bag-of-Words (BoW) model, a foundational technique in Natural Language Processing (NLP), offers a simplified way to represent documents, making the complex task of understanding and analyzing text data more manageable [8]. By employing this approach, the complex task of understanding and analyzing text data becomes more manageable. The text is split into words, meaningful phrases, or tokens. These tokens become the basic units of analysis and serve as features for representing the document. For each document, the frequency of occurrence of each token is calculated [10]. This results in a numerical representation, often in the form of a vector, where each element corresponds to the frequency of a particular token. By focusing on word frequencies, the BoW model captures the importance and prevalence of words within the document.

N-gram

In addition to individual words, we also consider the presence of n-grams within the article body. N-grams are contiguous sequences of n items (words in this context) from a given text [14]. Specifically, we focus on bigrams, which are pairs of adjacent words. By analyzing bigrams, we aim to capture the contextual information and relationships between adjacent words within the text. This enhances our understanding of the linguistic structures present in the articles and contributes to a more comprehensive feature representation.

Data Splitting and Feature Selection

To enhance the effectiveness of the stance detection model, we implement a strategic data splitting approach, dividing our dataset into training, validation, and testing

subsets with a 60:20:20 ratio. The 60:20:20 split ratio for training, validation, and testing sets is a commonly adopted practice in machine learning experiments. This allocation ensures that 60% of the data is dedicated to training the model, allowing it to learn and capture the underlying patterns within the data. The remaining 40% serves as validation set(20%) and testing set(20%). We utilize the train_test_split function from the sklearn.model_selection module to randomly split the data while maintaining the original distribution of classes. This split ratio was maintained consistently for both the lexical-based and TF-IDF based models. The code snippet below illustrates the data-splitting process:

```
# Split the data into training, validation, and testing sets
X = merged_df[['Headline', 'articleBody']]
y = merged_df['Stance']

#  split to create training, validation, and test sets
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)
```

Figure 4.4: Data Split for Training, Testing and Validation

The 'articleBody' column, enriched with lexical features, transforms a structured feature matrix. We utilize the DictVectorizer from scikit-learn, a versatile tool that efficiently converts the dictionary of lexical features into a numerical representation. This step is crucial for machine learning algorithms that require numerical input.

To further refine our feature set, we employ the chi-squared test for feature selection. This statistical test assesses the significance of each feature in distinguishing between different stance categories. We focus on the most informative and discriminative elements by selecting the top 1,000 features with the highest chi-squared statistics. The SelectKBest class from scikit-learn is used with the chi-squared test as the scoring function (chi2). This class allows us to select the top K features based on their chi-squared scores. The chi-squared test calculates a score for each feature, indicating the strength of its relationship with the target class. The score is computed based on the feature's presence or absence in each class and the expected frequencies.

```
# Select top 1000 most informative features using chi-squared test
selector = SelectKBest(score_func=chi2, k=1000)
X_train_selected = selector.fit_transform(X_train_sparse, y_train)
X_test_selected = selector.transform(X_test_sparse)
```

Figure 4.5: feature selection using the chi-squared test

Model Selection

Logistic regression was selected as the machine learning model for stance detection.

Two separate logistic regression models were trained: one using lexical features and the other using TF-IDF features.

Model Validation

Cross-validation tests were performed on both the lexical-based model and the TF–IDF model; the 5-fold cross-validation approach was used and it was done using the StratifiedKFold function in the sklearn. model_selection module. 5 folds were considered as reasonable because it is not sensible to make very large folds while at the same time we want to make sure that our model will perform well in a cross-validation study to estimate its performance. This strategy aids in eliminating wider deviation from the mean values of performance indicators that could have been realized in the sample. In splitting the dataset into 5 groups, we employed the principle of no-overlapping data where each fold was used for testing while the other four were used for training. The way the training and validation sets were being used here was that the first set was used for training while the second was for validation and this method ensured that all instances in the dataset were used twice and not once as is the common vice thus minimizing on the bias that might be present in a typical model evaluation. At each iteration, learning accuracy, precision, recall, and F1 scores on the validation set were computed based on the top-ranked items. These metrics proved useful in determining the extent to which instances could be classified accurately according to the tests done on the various models.

```
# Define the k-fold cross-validation strategy
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Figure 4.6: Model Validation: Evaluating Model Performance using Cross-Validation

Model Training and Evaluation

We train a logistic regression model using the selected features from the training set. Based on the extracted linguistic features, the model learns to predict the stance ('agree', 'disagree', 'discuss', 'unrelated'). We evaluate the model's performance on the testing set using various evaluation metrics, including accuracy, precision, recall, F1-score, and a detailed classification report.

4.4.2 TF-IDF Vectorization

TF-IDF (Term Frequency-Inverse Document Frequency) is a popular method for calculating word frequencies within a document. Term Frequency measures the occurrence of a term (word) in a document, indicating its relative frequency compared to the total number of words in that document. Inverse Document Frequency, on the other hand, assesses the rarity of a term across the entire dataset of documents. It assigns higher weights to infrequent terms across documents but appears frequently

within a specific document [31]. By combining Term Frequency and Inverse Document Frequency, TF-IDF provides a powerful representation of the importance of words in a document relative to the entire corpus. This technique is widely used in natural language processing tasks, such as text classification and information retrieval, to highlight the most distinctive and informative words or terms.

TF-IDF Score: The TF-IDF score for a word in a document is calculated as the product of its term frequency and inverse document frequency. This score reflects the importance of the word in the document relative to the entire corpus.

The formula for the TF-IDF Score is as follows: [21]

$$\text{TF}(t, d) = \frac{\text{Occurrence of } t \text{ in document } d}{\text{Total word frequency in document } d} \tag{4.1}$$

$$\text{IDF}(t, d) = \frac{\text{Count of all documents}}{\text{Count of all documents with term } t \text{ in } d} \tag{4.2}$$

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t, d) \tag{4.3}$$

Where:

- t represents a specific term or word in the vocabulary.
- d represents a specific document or text instance in the corpus.

To capture the significance of words within the context of the entire dataset, Term Frequency-Inverse Document Frequency (TF-IDF) was employed. This technique assigns higher weights to words that are informative and discriminative for the stance detection task. As a result, the textual data is transformed into a numerical representation where each feature (word) has a corresponding TF-IDF weight. We utilized the TfidfVectorizer class from scikit-learn to achieve this transformation. This process converts text data from the "Headline" and "articleBody" columns into TF-IDF weighted feature vectors. This transformation allows the Logistic Regression models to learn from the importance of words, based on their TF-IDF weights, and make predictions by distinguishing between different stances in the data.

Model Training and Evaluation

We train a logistic regression model with the TF-IDF weighted feature vectors obtained from the TfidfVectorizer. This model learns to predict the stance categories

('agree', 'disagree', 'discuss', 'unrelated') based on the TF-IDF weighted features extracted from the 'Headline' and 'articleBody' columns. The performance of the model is evaluated using various evaluation metrics, including accuracy, precision, recall, and F1-score, providing insights into its effectiveness in classifying stances.

Chapter 5

Results and Analysis

This section evaluates the performance of Logistic Regression models with different feature extraction techniques for stance detection. Two models were trained: one using lexical features and another using TF-IDF features. The TF-IDF model achieved superior performance compared to the lexical features model, with an accuracy of 81.75% compared to 77.82%. This highlights the importance of feature selection and representation in text classification tasks. TF-IDF features, by considering the context and distribution of words across documents, provided a more informative representation for the model to learn from.

Logistic Regression Model Performance with Lexical Features

The logistic regression model was trained using lexical features extracted from the text data. Lexical features include information such as word frequency and word complexity. The model achieved promising results, indicating its effectiveness in capturing patterns within the text data.

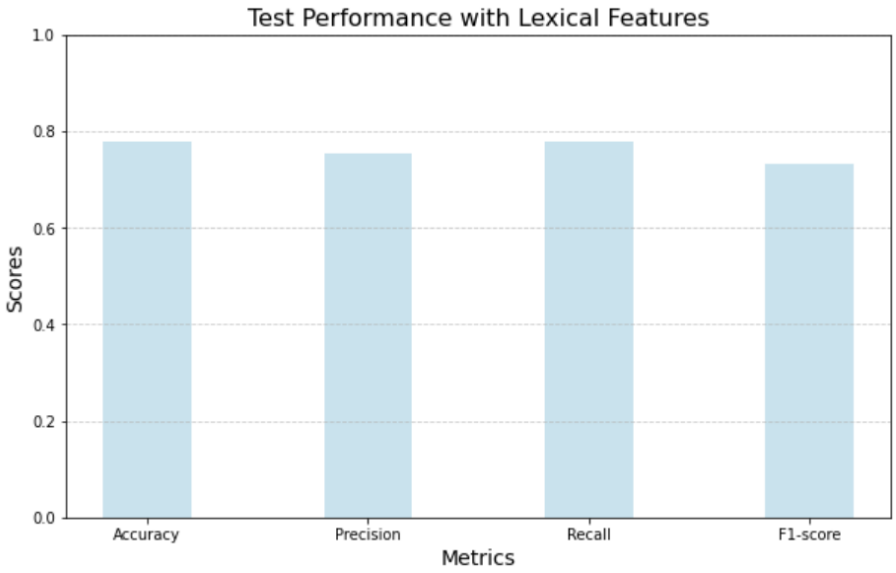

Figure 5.1: Performance graphs of lexical classifiers

Evaluation Metrics	Value
Accuracy	77.77%
Precision	75.20%
Recall	77.77%
F1-score	73.08%

Table 5.1: Logistic Regression Model Performance with Lexical Features

In Table 5.1, the model exhibited an accuracy of approximately 77.77%, demonstrating its ability to correctly classify the majority of instances. Precision, indicating the proportion of true positive predictions, was 75.20%, implying a relatively low number of false positives. Recall, representing the true positive rate, was 77.77%, suggesting that the model successfully identified a significant portion of the positive instances. The F1-score, a harmonic mean of precision and recall, was 73.08%, providing a balanced measure that considers both false positives and false negatives.

Logistic Regression Model Performance with TF-IDF

The logistic regression model was also evaluated using TF-IDF features, which capture the importance of each word in a document relative to a collection of documents. TF-IDF features take into account the frequency of words and their distribution across the corpus. The model demonstrated improved performance compared to lexical features.

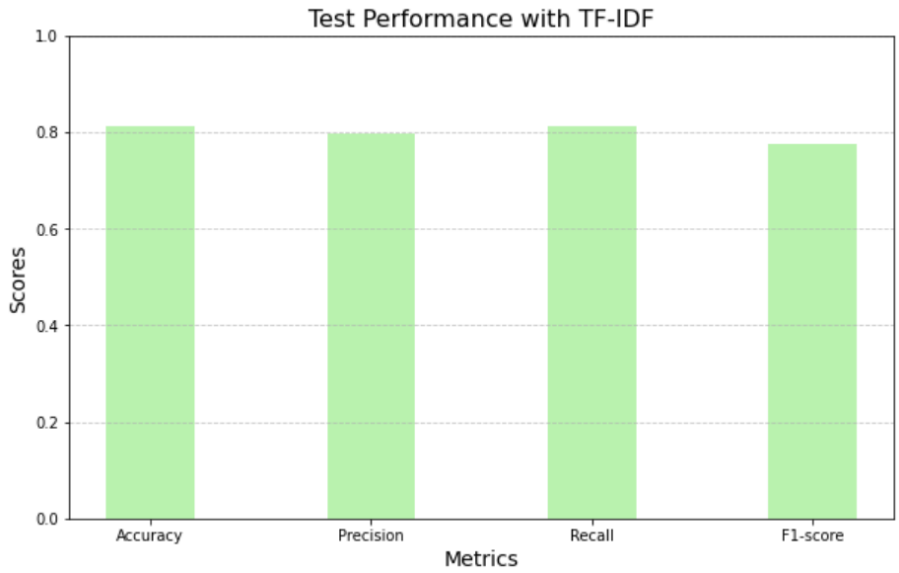

Figure 5.2: Performance graphs of TF-IDF classifiers

In Table 5.2, the model achieved an accuracy of approximately 81.06%, indicating a notable improvement over the lexical features model. Precision was recorded at

Evaluation Metrics	Value
Accuracy	81.06%
Precision	79.73%
Recall	81.06%
F1-score	77.49%

Table 5.2: Logistic Regression Model Performance with TF-IDF

79.73%, suggesting a higher proportion of true positive predictions. Recall maintained a high value of 81.06%, indicating the model's effectiveness in identifying positive instances. The F1-score increased to 77.49%, showcasing an improved balance between precision and recall. The results obtained from the logistic regression models highlight the importance of feature selection and representation in text classification tasks.

Classification Report for Lexical-based Model

The classification report provides a detailed breakdown of the model's performance for each class. In this case, the model is classifying text data into four categories: "agree," "disagree," "discuss," and "unrelated."

Class	Precision	Recall	F1-score	Support
agree	0.54	0.10	0.17	710
disagree	0.34	0.12	0.18	173
discuss	0.73	0.30	0.43	1777
unrelated	0.79	0.97	0.87	7335
Macro avg	0.60	0.38	0.41	9995
Weighted avg	0.75	0.78	0.73	9995

Table 5.3: Classification Report for lexical-based Model

In Table 5.3, the model exhibited varying performance across the classes. The "unrelated" class achieved the highest precision, recall, and F1 score, indicating the model's effectiveness in identifying unrelated text data. The "discuss" class also showed reasonable performance, with a precision of 0.73 and an F1-score of 0.43. However, the "agree" and "disagree" classes demonstrated lower recall and F1 scores, suggesting that the model struggled to correctly identify these classes. The macro and weighted averages provide an overall assessment of the model's performance, considering the contribution of each class.

Classification Report for TF-IDF Model

The classification report for the TF-IDF model shows improved performance compared to the lexical-based model, particularly for the "agree" and "discuss" classes.

Class	Precision	Recall	F1-score	Support
agree	0.73	0.16	0.27	710
disagree	0.52	0.08	0.13	173
discuss	0.77	0.44	0.56	1777
unrelated	0.82	0.98	0.89	7335
Macro avg	0.71	0.41	0.46	9995
Weighted avg	0.80	0.81	0.77	9995

Table 5.4: Classification Report for TF-IDF Model

In this report, the TF-IDF model demonstrated improved precision and F1-score for the "agree" class, indicating better identification of relevant instances. The "discuss" class also showed notable improvements in precision, recall, and F1-score, suggesting that the model is better able to distinguish between "discuss" instances. The "unrelated" class maintained its high performance, with precision, recall, and F1-score all above 0.80. The macro and weighted averages reflect the overall improvement in the model's performance, particularly in terms of F1-score.

The TF-IDF model appears to offer more balanced performance across the classes, while the lexical-based model struggles with certain classes, specifically "agree" and "disagree." These findings can guide further model refinement and feature engineering to enhance classification accuracy for all classes.

Chapter 6

Discussion

RQ1: In the context of tackling misinformation and upholding information integrity, how do logistic regression and the TF-IDF model compare in their performance and effectiveness in classifying the stance of news articles and sentences using lexical features for fake news detection systems?

Answer To address the issue of misinformation and maintain information integrity, logistic regression, and the TF-IDF model were compared in terms of their performance and effectiveness in classifying the stance of news articles and sentences using lexical features for fake news detection systems.

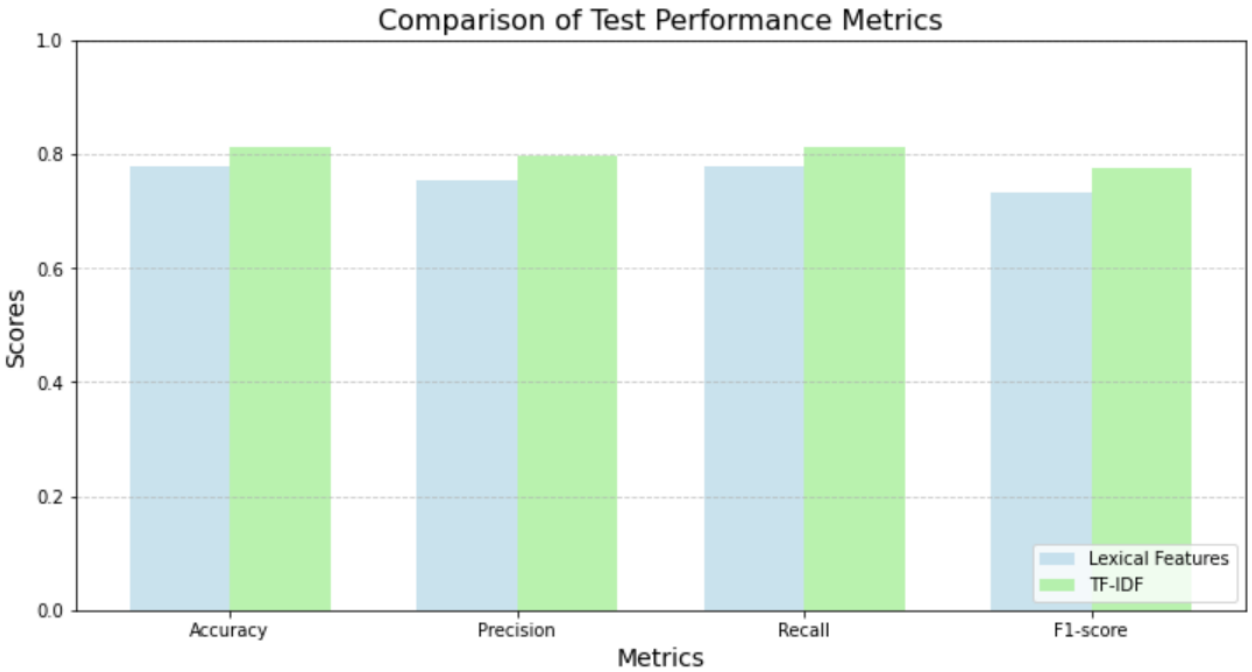

Figure 6.1: Performance Comparison of Lexical and TF-IDF Models

Accuracy

The TF-IDF Features Model demonstrated a significant improvement in accuracy (81.06%) compared to the Lexical Features Model (77.77%). This translates to cor-

rectly classifying approximately 81.06% and 77.77% of instances across all stances (agree, disagree, discuss, unrelated).

Precision

The Lexical Features Model, precision stands at 75.20%, suggesting the inclusion of some irrelevant instances in stance classifications, whereas the TF-IDF Features Model exhibits an improved precision of 79.73%.

Recall

The Lexical Features Model achieves a recall of 77.77%, capturing a substantial portion of true instances within each stance category. In contrast, the TF-IDF Features Model demonstrates an improved recall of 81.06%, highlighting its enhanced ability to identify a higher proportion of true instances within each stance.

F1-score

The Lexical Features model achieved an F1-score of 73.08%, providing a balanced view of its performance for all stance categories. However, The TF-IDF Features model exhibited a slight improvement, reaching an F1-score of 77.49%. This suggests that while both models performed adequately, the TF-IDF features enabled a more balanced performance in stance classification.

6.1 Confusion Matrix Heatmap for Lexical-based Model

The heatmap of the Lexical Features-based model displayed below in Figure 6.2 shows how well the model categorizes news articles into four groups; agree, disagree, discuss, and unrelated. It's, like a 4x4 grid where each cell (i, j) shows when the actual class is i and the predicted class is j.

Labels on Axes

- X axis (Predicted); Shows the models predicted classes. 'agree' 'disagree' 'discuss and 'unrelated'.

- Y axis (Actual); Represents the classes in the dataset matching those on the X axis.

Cells: Each cell holds a number indicating instances for a predicted class pair. Cells along the way show classified instances for each class. Off-diagonal cells indicate misclassifications.

Color Gradient: The color intensity in each cell reflects instance counts. Shades for counts and lighter shades, for lower counts.Understanding

Diagonal Components: The diagonal elements play a role as they represent the

accuracy of predictions made for each category. For instance, a high value, in the cell at (0,0) indicates that the model accurately predicted instances of 'agree' as 'agree.'

Off Diagonal Components: These elements reveal where the model has made mistakes. For example, if the value in the cell at (0,1) is high, many instances labeled as 'agree' were inaccurately predicted as 'disagree.' Analyzing these cells helps pinpoint areas where the model encounters challenges.

Performance by Class: By examining both the rows and columns of the confusion matrix, we can evaluate how well the model performs for each category. For instance, the 'unrelated' class has high values in its row but low values in its column, it indicates that 'unrelated' instances are often misclassified as other classes.

Example Analysis

Agree Class:

- The 'agree' class has a high count in the (0,0) cell, it indicates good performance in predicting 'agree' instances.
- There are significant counts in cells (0,1), (0,2), or (0,3), which shows that 'agree' instances are often misclassified as 'disagree', 'discuss', or 'unrelated'.

Disagree Class: A similar analysis can be done for the 'disagree' class by looking at the (1,1) cell and its corresponding row and column.

Discuss Class: The (2,2) cell and its row and column provide insights into the model's performance for the 'discuss' class.

Unrelated Class: The (3,3) cell and its row and column indicate how well the model predicts 'unrelated' instances.

The confusion matrix heatmap for the Lexical Features-based model provides a comprehensive view of the model's performance. We can identify strengths and weaknesses in the model's predictions by analyzing the diagonal and off-diagonal elements. This analysis helps in understanding where the model performs well and where it needs improvement, guiding further model tuning and feature engineering efforts.

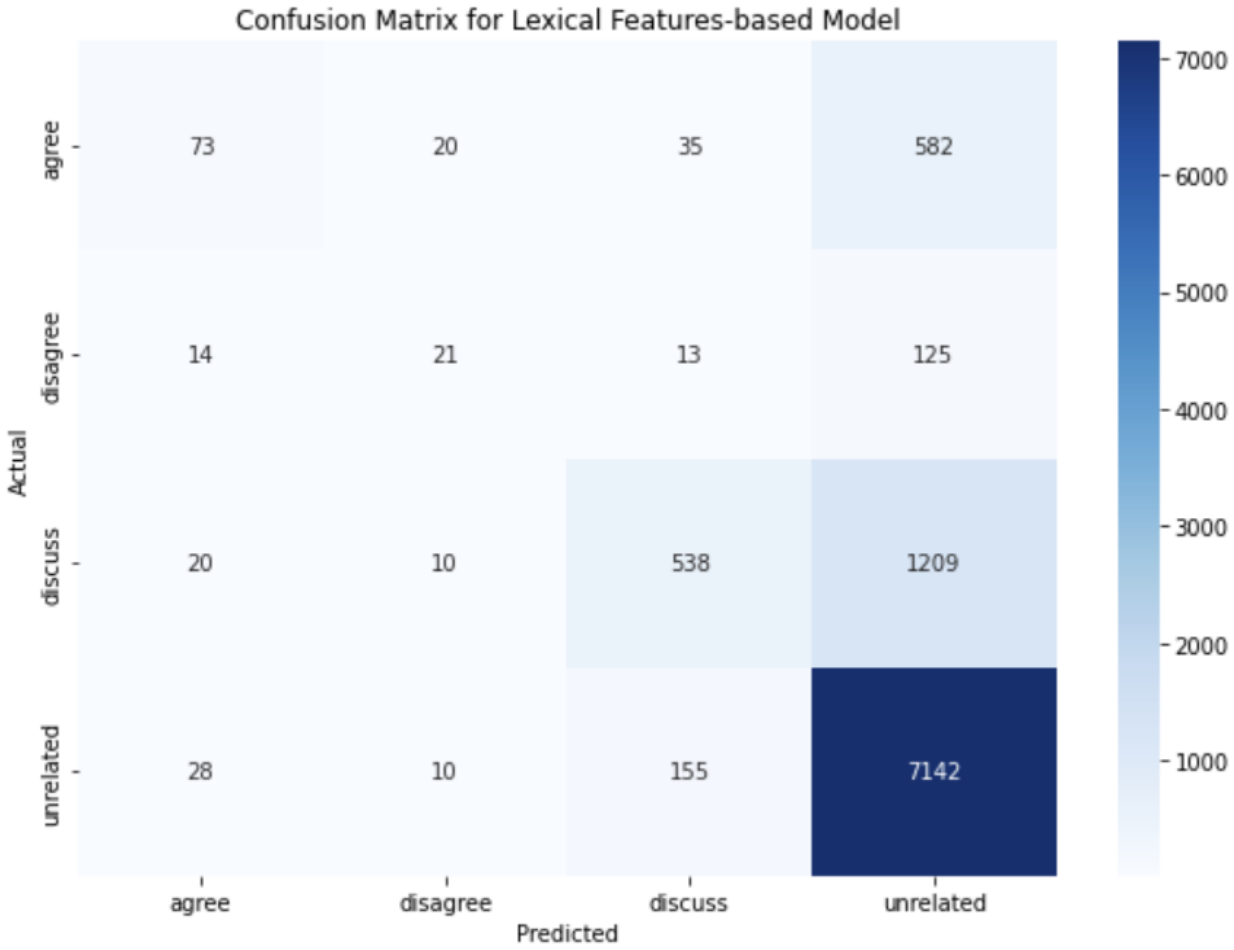

Figure 6.2: Heatmap illustrating the confusion matrix for the logistic regression model trained with lexical features

6.2 Confusion Matrix Heatmap for TF-IDF-based Model

The confusion matrix heatmap for the TF-IDF-based model also represents the performance of the model in classifying news articles into the same four categories: agree, disagree, discuss, and unrelated. The structure of the heatmap is similar to that of the Lexical Features-based model, but the performance metrics are expected to differ.

Labels on Axes

- X-axis (Predicted): Represents the predicted classes by the model.
- Y-axis (Actual): Represents the actual classes in the dataset.

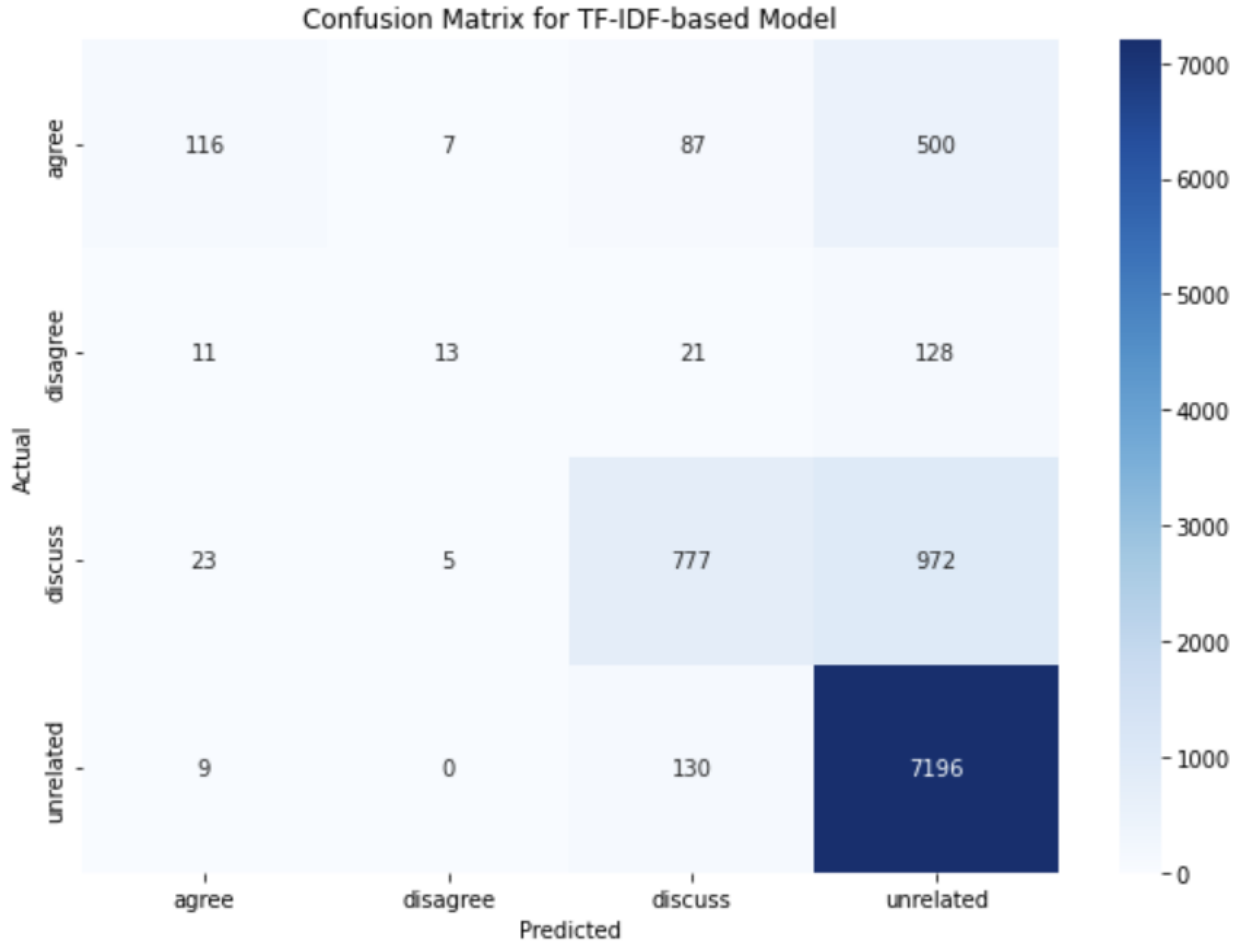

Figure 6.3: Heatmap illustrating the confusion matrix for the logistic regression model trained with TF-IDF features

Cells: Each cell shows the count of instances for a specific combination of actual and predicted classes. Diagonal cells represent correct predictions, while off-diagonal cells represent misclassifications.

Color Intensity: The color intensity indicates the magnitude of the count, with darker shades representing higher counts.

Diagonal Elements: The diagonal elements indicate the number of correct predictions for each class. A high value in these cells suggests good performance for that class.

Off-Diagonal Elements: These elements show where the model made errors. Analyzing these cells helps identify specific areas where the model struggles.

Class-wise Performance: By examining the rows and columns, we can assess the model's performance for each class. For example, if the 'unrelated' class has high values in its row but low values in its column, it indicates that 'unrelated' instances are often misclassified as other classes.

Example Analysis

Agree Class: If the 'agree' class has a high count in the (0,0) cell, it indicates good performance in predicting 'agree' instances. If there are significant counts in cells (0,1), (0,2), or (0,3), it shows that 'agree' instances are often misclassified as 'disagree', 'discuss', or 'unrelated'.

Disagree Class: A similar analysis can be done for the 'disagree' class by looking at the (1,1) cell and its corresponding row and column.

Discuss Class: The (2,2) cell and its row and column provide insights into the model's performance for the 'discuss' class.

Unrelated Class: The (3,3) cell and its row and column indicate how well the model predicts 'unrelated' instances.

Comparison with Lexical Features-based Model

Accuracy: The TF-IDF-based model is expected to have higher accuracy, as the overall performance metrics indicate. This should be reflected in higher counts along the diagonal cells compared to the Lexical Features-based model.

Misclassifications: The off-diagonal elements should show fewer misclassifications, indicating better performance in distinguishing between classes.

Class-wise Improvements: Specific classes, such as 'unrelated', may show significant improvements in correct predictions, reducing the counts in off-diagonal cells.

The confusion matrix heatmap for the TF-IDF-based model provides a detailed view of the model's performance. We can identify improvements and areas where the TF-IDF model excels by comparing it with the lexical features-based model. This analysis helps understand the TF-IDF representation's effectiveness in enhancing the model's ability to classify news articles accurately, guiding further model development and optimization efforts.

Confusion Matrix for Logistic Regression with Lexical Features:

- The confusion matrix is a table that summarizes the performance of a classification model. It evaluates the performance of a logistic regression model with lexical features.

- The rows of the matrix represent the true labels of the test data, and the columns represent the predicted labels by the model.

- The values in the matrix indicate the number of samples that fall into each category. For example, the value in the "agree-agree" cell represents the number

of samples that were labeled as "agree" and were correctly predicted as "agree" by the model.

- The diagonal elements of the matrix represent the number of correctly classified samples for each class. For example, the value in the "agree-agree" cell represents the true positives for the "agree" class.

- Off-diagonal elements represent misclassifications. For example, the value in the "agree-disagree" cell represents the number of samples that were actually "agree" but were misclassified as "disagree" by the model.

- The confusion matrix provides insights into the model's performance, such as its ability to distinguish between different classes, the presence of any class imbalances, and the types of errors the model is making.

- By analyzing the confusion matrix, we can calculate various performance metrics such as accuracy, precision, recall, and F1-score for each class.

Confusion Matrix for Logistic Regression with Lexical Features:

	agree	disagree	discuss	unrelated
agree	73	20	35	582
disagree	14	21	13	125
discuss	20	10	538	1209
unrelated	28	10	155	7142

Figure 6.4: Confusion Matrix for Logistic Regression Model with Lexical Features, showcasing Agree, Disagree, Discuss, Unrelated

Confusion Matrix for Logistic Regression with TF-IDF:

- This confusion matrix evaluates the performance of a logistic regression model that uses TF-IDF features instead of lexical features.

- Similar to the previous matrix, the rows represent the true labels, the columns represent the predicted labels, and the values indicate the number of samples in each category.

- By comparing the confusion matrices of the two models, we can assess the impact of using different feature representations on the classification performance.

- The confusion matrix for TF-IDF features may show different patterns of correct and incorrect predictions compared to the lexical feature matrix. This comparison can help us understand which feature representation is more effective for the given classification task.

- The confusion matrix allows us to identify any specific classes that the model struggles with or any patterns of misclassification that are unique to the TF-IDF model.

Confusion Matrix for Logistic Regression with TF-IDF:

	agree	disagree	discuss	unrelated
agree	116	7	87	500
disagree	11	13	21	128
discuss	23	5	777	972
unrelated	9	0	130	7196

Figure 6.5: Confusion Matrix for Logistic Regression Model with TF-IDF Features, showcasing Agree, Disagree, Discuss, Unrelated

6.3 Reflection

We highlighted the research gap concerning the integration of TF-IDF and lexical-based stance detection techniques with logistic regression for fake news detection. While previous studies have explored these techniques individually, there has been limited research combining them within a unified framework. This thesis aimed to bridge this gap by investigating the interplay between TF-IDF, Lexical-Based Stance Detection, and Logistic Regression for classifying the stance of news articles and sentences.

Through our experimental approach, we compared the performance of two logistic regression models: one trained with lexical features extracted from the text data and another trained with TF-IDF weighted features. The results demonstrated the TF-IDF model, achieving an accuracy of 81.75% compared to 77.82% for the lexical features model.

The TF-IDF model's improved performance can be attributed to its ability to capture the importance and contextual relevance of words within news articles and headlines. By weighing the significance of terms based on their frequency and distribution across the corpus, the TF-IDF representation provided a more informative feature set for the logistic regression model to learn from.

Furthermore, the classification reports revealed that the TF-IDF model exhibited

better performance in distinguishing between specific stance categories, such as "agree" and "discuss." This suggests that the TF-IDF representation effectively captured the nuances and linguistic patterns associated with these stances, enabling a more accurate classification.

While the lexical features model also demonstrated promising results, its performance was hindered by certain classes, indicating potential limitations in capturing the complexities of stance detection solely through word frequencies and lexical patterns.

By integrating TF-IDF and Lexical-Based Stance Detection techniques within a Logistic Regression framework, this thesis contributes to the development of more robust and effective fake news detection systems. The findings highlight the importance of feature selection and representation in text classification tasks and provide valuable insights into the power of TF-IDF in capturing the nuances of textual data for stance detection.

This thesis addresses the identified research gap by comprehensively comparing TF-IDF and Lexical-Based Stance Detection techniques within a Logistic Regression framework for fake news detection. The findings contribute to the ongoing efforts to combat misinformation and promote information integrity, while also providing a foundation for future research in this critical field.

Chapter 7

Conclusions and Future Work

7.1 Conclusion

This thesis presents a comprehensive study of stance detection in textual data using machine learning techniques. We explored the effectiveness of logistic regression models with different feature representations, namely lexical features and TF-IDF features. The results obtained from our experiments provide valuable insights into the performance of these models and the importance of feature selection and representation in text classification tasks.

The lexical-based approach, utilizing word frequency, word complexity, and stance-related word occurrence, achieved promising results with an accuracy of 77.82%. This model effectively captured patterns within the text data, as evident from its precision, recall, and F1-score metrics. However, we observed that the model struggled with certain classes, particularly "agree" and "disagree," as indicated by the classification report. This suggests room for improvement in distinguishing between these specific stances.

On the other hand, the TF-IDF-based model demonstrated superior performance, achieving an accuracy of 81.75%. TF-IDF features take into account the context and distribution of words across documents, providing a more informative representation for the model to learn from. The improved precision, recall and F1-score for the "agree" and "discuss" classes highlight the effectiveness of TF-IDF in capturing the importance of words within the text.

7.2 Future Work

- Exploring additional lexical and semantic features could further improve the model's performance. For instance, incorporating word embeddings or syntactic features may capture more nuanced linguistic patterns.

- Employ ensemble techniques, such as stacking, boosting, or bagging, to combine multiple models and improve overall stance detection accuracy.

- As observed in the classification reports, class imbalance may impact model performance, particularly for minority classes like "disagree." Techniques such

as oversampling, undersampling, or using class weights can mitigate this issue.

- Optimal model performance may be achieved through systematic hyperparameter tuning. Techniques such as grid search or Bayesian optimization can help identify the best combination of parameters for logistic regression models.

- Optimize the models for speed and efficiency to enable real-time stance detection, making it applicable for social media monitoring, customer feedback analysis, and sentiment analysis in dynamic environments.

References

[1] "Array programming with NumPy | Nature." [Online]. Available: https://www.nature.com/articles/s41586-020-2649-2

[2] "Fake News Challenge." [Online]. Available: https://www.kaggle.com/datasets/abhinavkrjha/fake-news-challenge

[3] "Fake News Challenge." [Online]. Available: http://www.fakenewschallenge.org/

[4] "Matplotlib and Seaborn | SpringerLink." [Online]. Available: https://link.springer.com/chapter/10.1007/978-1-4842-4470-8_12

[5] "sklearn.feature_extraction.DictVectorizer." [Online]. Available: https://scikit-learn/stable/modules/generated/sklearn.feature_extraction.DictVectorizer.html

[6] "sklearn.feature_selection.chi2." [Online]. Available: https://scikit-learn/stable/modules/generated/sklearn.feature_selection.chi2.html

[7] "Stopwords in technical language processing | PLOS ONE." [Online]. Available: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0254937

[8] "Bag-of-words model," Apr. 2024, page Version ID: 1217165620. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Bag-of-words_model&oldid=1217165620

[9] "Chi-squared distribution," Apr. 2024, page Version ID: 1218540953. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Chi-squared_distribution&oldid=1218540953

[10] V. Agarwal, H. P. Sultana, S. Malhotra, and A. Sarkar, "Analysis of Classifiers for Fake News Detection," *Procedia Computer Science*, vol. 165, pp. 377–383, Jan. 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050920300430

[11] B. Al Asaad and M. Erascu, "A Tool for Fake News Detection," in *2018 20th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*, Sep. 2018, pp. 379–386. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8750741

[12] N. Ari and M. Ustazhanov, "Matplotlib in python," in *2014 11th International Conference on Electronics, Computer and Computation (ICECCO)*, Sep. 2014, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/6997585

[13] R. Betancourt and S. Chen, "pandas Library," in *Python for SAS Users: A SAS-Oriented Introduction to Python*, R. Betancourt and S. Chen, Eds. Berkeley, CA: Apress, 2019, pp. 65–109. [Online]. Available: https://doi.org/10.1007/978-1-4842-5001-3_3

[14] W. B. Cavnar and J. M. Trenkle, "N-Gram-Based Text Categorization."

[15] W. Cheng and E. Hüllermeier, "Combining instance-based learning and logistic regression for multilabel classification," *Machine Learning*, vol. 76, no. 2, pp. 211–225, Sep. 2009. [Online]. Available: https://doi.org/10.1007/s10994-009-5127-5

[16] Computer Science Department, Faculty of Information Technology, Al-Ahliyya Amman University, Amman, Jordan and M. M. Al-Tahrawi, "Arabic Text Categorization Using Logistic Regression," *International Journal of Intelligent Systems and Applications*, vol. 7, no. 6, pp. 71–78, May 2015. [Online]. Available: http://www.mecs-press.org/ijisa/ijisa-v7-n6/v7n6-8.html

[17] J. Hao and T. K. Ho, "Machine Learning Made Easy: A Review of Scikit-learn Package in Python Programming Language," *Journal of Educational and Behavioral Statistics*, vol. 44, no. 3, pp. 348–361, Jun. 2019, publisher: American Educational Research Association. [Online]. Available: https://doi.org/10.3102/1076998619832248

[18] F. M. Hassan and M. Lee, "Multi-stage News-Stance Classification Based on Lexical and Neural Features," in *13th International Conference on Computational Intelligence in Security for Information Systems (CISIS 2020)*, Herrero, C. Cambra, D. Urda, J. Sedano, H. Quintián, and E. Corchado, Eds. Cham: Springer International Publishing, 2021, pp. 218–228.

[19] K. Jindal, V. Bhardwaj, S. Ray, U. Parvez, and V. Raj, "Compare The Performance of Machine Learning Classifiers for Misinformation Detection," in *2022 5th International Conference on Contemporary Computing and Informatics (IC3I)*, Dec. 2022, pp. 1284–1289. [Online]. Available: https://ieeexplore.ieee.org/document/10072306

[20] Z. Khanam, B. N. Alwasel, H. Sirafi, and M. Rashid, "Fake News Detection Using Machine Learning Approaches," *IOP Conference Series: Materials Science and Engineering*, vol. 1099, no. 1, p. 012040, Mar. 2021. [Online]. Available: https://iopscience.iop.org/article/10.1088/1757-899X/1099/1/012040

[21] V. Kumar, A. Kumar, A. K. Singh, and A. Pachauri, "Fake News Detection using Machine Learning and Natural Language Processing," in *2021 International Conference on Technological Advancements and Innovations (ICTAI)*. Tashkent, Uzbekistan: IEEE, Nov. 2021, pp. 547–552. [Online]. Available: https://ieeexplore.ieee.org/document/9673378/

[22] V. Kumar and B. Subba, "A TfidfVectorizer and SVM based sentiment analysis framework for text data corpus," in *2020 National Conference on Communications (NCC)*, Feb. 2020, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9056085

[23] E. Loper and S. Bird, "NLTK: The Natural Language Toolkit," May 2002, arXiv:cs/0205028. [Online]. Available: http://arxiv.org/abs/cs/0205028

[24] B. Mahesh, *Machine Learning Algorithms -A Review*, Jan. 2019.

[25] V. Mohan, "Text Mining: Open Source Tokenization Tools: An Analysis," vol. 3, pp. 37–47, Jan. 2016.

[26] D. Mrowca, E. Wang, and A. Kosson, "Stance Detection for Fake News Identification."

[27] R. Oshikawa, J. Qian, and W. Y. Wang, "A Survey on Natural Language Processing for Fake News Detection," Mar. 2020, arXiv:1811.00770 [cs]. [Online]. Available: http://arxiv.org/abs/1811.00770

[28] C. D. Paice, "An Evaluation Method for Stemming Algorithms," in *SIGIR '94*, B. W. Croft and C. J. Van Rijsbergen, Eds. London: Springer London, 1994, pp. 42–50. [Online]. Available: http://link.springer.com/10.1007/978-1-4471-2099-5_5

[29] R. L. Plackett, "Karl Pearson and the Chi-Squared Test," *International Statistical Review / Revue Internationale de Statistique*, vol. 51, no. 1, pp. 59–72, 1983, publisher: [Wiley, International Statistical Institute (ISI)]. [Online]. Available: https://www.jstor.org/stable/1402731

[30] K. Poddar, G. B. Amali D., and K. Umadevi, "Comparison of Various Machine Learning Models for Accurate Detection of Fake News," in *2019 Innovations in Power and Advanced Computing Technologies (i-PACT)*. Vellore, India: IEEE, Mar. 2019, pp. 1–5. [Online]. Available: https://ieeexplore.ieee.org/document/8960044/

[31] S. Qaiser and R. Ali, "Text Mining: Use of TF-IDF to Examine the Relevance of Words to Documents," *International Journal of Computer Applications*, vol. 181, Jul. 2018.

[32] D. M. Rajeswari, "FAKE NEWS DETECTION USING LOGISTIC REGRESSION ALGORITHM WITH MACHINE LEARNING," vol. 9, no. 6, 2022.

[33] J. Shaikh and R. Patil, "Fake News Detection using Machine Learning," in *2020 IEEE International Symposium on Sustainable Energy, Signal Processing and Cyber Security (iSSSC)*. Gunupur Odisha, India: IEEE, Dec. 2020, pp. 1–5. [Online]. Available: https://ieeexplore.ieee.org/document/9358890/

[34] A. Shete, H. Soni, Z. Sajnani, and A. Shete, "Fake News Detection Using Natural Language Processing and Logistic Regression," in *2021 2nd International Conference on Advances in Computing, Communication, Embedded and Secure Systems (ACCESS)*, Sep. 2021, pp. 136–140. [Online]. Available: https://ieeexplore.ieee.org/document/9563292

[35] P. P. Shinde and S. Shah, "A Review of Machine Learning and Deep Learning Applications," in *2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBEA)*, Aug. 2018, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8697857

[36] S. Shrivastava, R. Singh, C. Jain, and S. Kaushal, "A Research on Fake News Detection Using Machine Learning Algorithm," in *Smart Systems: Innovations in Computing*, A. K. Somani, A. Mundra, R. Doss, and S. Bhattacharya, Eds. Singapore: Springer, 2022, pp. 273–287.

[37] V. Simaki, C. Paradis, and A. Kerren, "A two-step procedure to identify lexical elements of stance constructions in discourse from political blogs," *Corpora*, vol. 2019, Nov. 2019.

[38] N. L. Siva Rama Krishna and M. Adimoolam, "Fake News Detection System using Logistic Regression and Compare Textual Property with Support Vector Machine Algorithm," in *2022 International Conference on Sustainable Computing and Data Communication Systems (ICSCDS)*, Apr. 2022, pp. 48–53. [Online]. Available: https://ieeexplore.ieee.org/document/9760768

[39] M. Umer, Z. Imtiaz, S. Ullah, A. Mehmood, G. S. Choi, and B.-W. On, "Fake News Stance Detection Using Deep Learning Architecture (CNN-LSTM)," *IEEE Access*, vol. 8, pp. 156 695–156 706, 2020, conference Name: IEEE Access. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9178321

[40] H. C. Wu, R. W. P. Luk, K. F. Wong, and K. L. Kwok, "Interpreting TF-IDF term weights as making relevance decisions," *ACM Transactions on Information Systems*, vol. 26, no. 3, pp. 13:1–13:37, Jun. 2008. [Online]. Available: https://doi.org/10.1145/1361684.1361686

[41] S.-J. Yen, Y.-S. Lee, J.-C. Ying, and Y.-C. Wu, "A logistic regression-based smoothing method for Chinese text categorization," *Expert Systems with Applications*, vol. 38, no. 9, pp. 11 581–11 590, Sep. 2011. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417411004441

[42] X. Zhang and A. A. Ghorbani, "An overview of online fake news: Characterization, detection, and discussion," *Information Processing & Management*, vol. 57, no. 2, p. 102025, Mar. 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0306457318306794

Faculty of Faculty, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden